Project 3: Pose Estimation - PoseCNN

PoseCNN: A Classic end-to-end pose estimation network

In this exercise you will implement an **end-to-end** object pose estimator, based on PoseCNN, which consists of two stages - feature extraction with a backbone network and pose estimation represented by instance segmentation, 3D translation estimation, and 3D rotation estimation. We will train it to estimate the pose of a set of object classes and evaluate the estimation accuracy.

Getting Started

Setup Eviroment

Before getting started, we need to run some boilerplate code to set up our environ
ment, same as previous assignments. You'll need to rerun this setup code each time you start the notebook.

Frist we install a couple packages to help with processing and visualizing object models. Refer to enviroment.yml to package we need to install. Whats more, you can try

```
conda env create -f environment.yml
```

to create the environment with required package installed instantly!

Once you have successfully mounted your Google Drive and located the path to this assignment, run the following cell to allow us to import from the `.py` files of this assignment. If it works correctly, it should print the message:

```
Hello from pose_cnn.py!
Hello from p3_helper.py!
```

as well as the last edit time for the file `pose_cnn.py`.

```
import os
import time

from pose_cnn import hello_pose_cnn
from p3_helper import hello_helper


hello_pose_cnn()
hello_helper()
GOOGLE_DRIVE_PATH = '.'
pose_cnn_path = os.path.join(GOOGLE_DRIVE_PATH, "pose_cnn.py")
pose_cnn_edit_time = time.ctime(
    os.path.getmtime(pose_cnn_path)
)
print("pose_cnn.py last edited on %s" % pose_cnn_edit_time)
```

```
Hello from pose_cnn.py!
Hello from P3_helper.py!
pose_cnn.py last edited on Fri Jan 10 21:38:18 2025
```

Load several useful packages that are used in this notebook:

```
import os
import time

import matplotlib.pyplot as plt
import torch
import torchvision

%matplotlib inline

from p3_helper import *
from utils import reset_seed
from grad import rel_error

# for plotting
plt.rcParams["figure.figsize"] = (10.0, 8.0)  # set default size of plots
plt.rcParams["font.size"] = 16
plt.rcParams["image.interpolation"] = "nearest"
plt.rcParams["image.cmap"] = "gray"
```

We will use GPUs to accelerate our computation in this notebook. Run the following to make sure GPUs are enabled:

```
if torch.cuda.is_available():
    print("Good to go!")
    DEVICE = torch.device("cuda")
else:
    print("Please set GPU via Edit -> Notebook Settings.")
    DEVICE = torch.device("cpu")
```

```
Good to go!
```

Load PROPS Pose Dataset

During the majority of our homework assignments so far, we have used the PROPS Classification or Detection datasets for image processing tasks.

In order to train and evaluate object pose estimation models, we need a dataset where each image is annotated with a *set* of *pose labels*, where each pose label gives the 3DoF position and 3DoF orientation of some object in the image.

We will use the PROPS Pose dataset, which provides annotations of this form.
Our PROPS Detection dataset is much smaller than typical benchmarking pose estimation datasets, and thus easier to manage in an homework assignment.
PROPS comprises annotated bounding boxes for 10 object classes:
`["master_chef_can", "cracker_box", "sugar_box", "tomato_soup_can", "mustard_bottle", "tuna_fish_can", "gelatin_box", "potted_meat_can", "mug", "large_marker"]`.

The choice of these objects is inspired by the YCB object and Model set commonly used in robotic perception models.

We create a `PyTorch Dataset` class named `PROPSPoseDataset` in `PROPSPoseDataset.py` that will download the PROPS Pose dataset.

Run the following two cells to set a few config parameters and then download the train/val sets for the PROPS Pose dataset.

```
import multiprocessing

# Set a few constants related to data loading.
NUM_CLASSES = 10
BATCH_SIZE = 4
NUM_WORKERS = multiprocessing.cpu_count()
```

```
from PROPSPoseDataset import PROPSPoseDataset

DATA_PATH = "./data"
train_dataset = PROPSPoseDataset(
    DATA_PATH, "train",
    download=False  # True (for the first time)
)
val_dataset = PROPSPoseDataset(DATA_PATH, "val")

print(f"Dataset sizes: train ({len(train_dataset)}), val ({len(val_dataset)})")
```

```
Dataset sizes: train (500), val (500)
```

This dataset will format each sample from the dataset as a dictionary containing the following keys:

- 'rgb': a numpy float32 array of shape (3, 480, 640) scaled to range [0,1]
- 'depth': a numpy int32 array of shape (1, 480, 640) in (mm)
- 'objs_id': a numpy uint8 array of shape (10,) containing integer ids for visible objects (1-10) and invisible objects (0)
- 'label': a numpy bool array of shape (11, 480, 640) containing instance segmentation for objects in the scene
- 'bbx': a numpy float64 array of shape (10, 4) containing (x, y, w, h) coordinates of object bounding boxes
- 'RTs': a numpy float64 array of shape (10, 3, 4) containing homogeneous transformation matrices per object into camera coordinate frame
- 'centermaps': a numpy float64 array of shape (30, 480, 640) containing (dx, dy, z) coordinates to each object's centroid
- 'centers': a numpy float64 array of shape (10, 2) containing (x, y) coordinates of object centroids projected to image plane

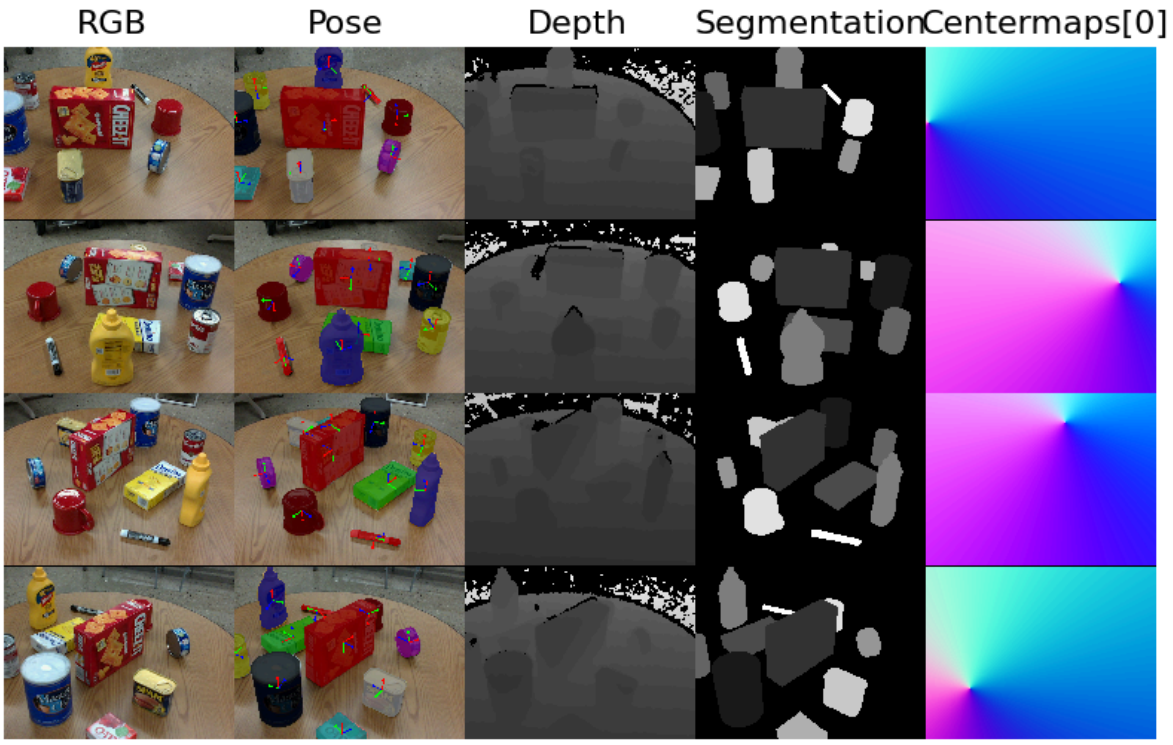
This dataset assumes that the upper left of the image is the origin point (0, 0).

Visualize Dataset

Now let's visualize a few samples from our validation set to make sure the images and labels are loaded correctly. In this next cell, we'll use the `visualize_dataset` function from `utils.py` to view the RGB observation and labeled pose labels for three random samples.

In the below figure, the final column plots the centermaps for class 0, which corresponds to the master chef coffee can. This plot is included to give a sense of how the centermaps represent gradients towards the object's centroid.

```
from utils import reset_seed, visualize_dataset

reset_seed(0)

grid_vis = visualize_dataset(val_dataset,alpha = 0.25)
plt.axis('off')
plt.imshow(grid_vis)
plt.show()
```



Implementing PoseCNN

Now that we have our dataset loaded and ready to use, we'll begin implementing a variant of the PoseCNN network. This architecture is designed to take an RGB color image as input and produce a 6 degrees-of-freedom pose estimate for each instance of an object within the scene from which the image was taken. To do this, PoseCNN uses 5 operations within the architecture. First, a backbone convolutional feature extraction network is used to produce a tensor representing learned features from the input image. Second, the extracted features are processed by an embedding branch to reduce the spatial resolution and memory overhead for downstream layers. Third, an instance segmentation branch uses the embedded features to identify regions in the image corresponding to each object instance (regions of interest). Fourth, the translations for each

object instance are estimated using a translation branch along with the embedded features. Finally, a rotation branch uses the embedded features to estimate a rotation, in the form of a quaternion, for each region of interest.

Thr architecture is shown in more detail from Figure 2 of the PoseCNN paper:

Now, we will implement a variant of this architecture that performs each of the 5 operations using PyTorch and data from our `PROPSPoseDataset`. The remainder of the features for this project will be implemented in the `pose_cnn.py` file.

Implementing the Backbone and Feature Extraction Branch

In the past project, you implemented a deep convolutional network from scratch. In this project, we'll use torchvision's pretrained convolutional networks for our backbone convolutional feature extractor. Specifically, we'll use the VGG16 model as our feature extractor:

```
import torchvision.models as models

vgg16 = models.vgg16(weights=models.VGG16_Weights.IMAGENET1K_V1)
```

We can inspect the definition of this model by referring to the model's source code within torchvision. From this definition, we can see that the `vgg16.features` variable stores the feature extraction portion of VGG16 before average pooling. We'll use this pretrained VGG16 model for our PoseCNN model's pretrained extraction backbone. Print out the layers in `vgg16.features` by running the code cell below:

```
vgg16.features
```

```
Sequential(
  (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (1): ReLU(inplace=True)
  (2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (3): ReLU(inplace=True)
  (4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (5): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (6): ReLU(inplace=True)
  (7): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (8): ReLU(inplace=True)
  (9): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (10): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (11): ReLU(inplace=True)
  (12): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (13): ReLU(inplace=True)
  (14): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (15): ReLU(inplace=True)
  (16): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
  (17): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
```

```
  (18): ReLU(inplace=True)
  (19): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (20): ReLU(inplace=True)
  (21): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (22): ReLU(inplace=True)
  (23): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
  (24): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (25): ReLU(inplace=True)
  (26): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (27): ReLU(inplace=True)
  (28): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (29): ReLU(inplace=True)
  (30): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
)
```

Using this `vgg16` model as the `pretrained_network`, we implement PoseCNN's feature extraction branch for you in the `FeatureExtraction` class of `pose_cnn.py`. You should inspect this function to understand how the output `feature1` and `feature2` tensors are created. In addition, observe that we freeze the early layers of our pretrained network by setting the corresponding convolutional weights and bias to have `requires_grad=False`. Freezing these layers will speed up training and help reduce overfitting.

We can inspect the expected sizes of `feature1` and `feature2` by passing the `FeatureExtraction` dummy input:

```
import torchvision.models as models
from pose_cnn import FeatureExtraction

# Based on PoseCNN section III.B, the output features should
# be 1/8 and 1/16 the input's spatial resolution with 512 channels
print('feature1 expected shape: (N, {}, {}, {})'.format(512, 480//8, 640//8))
print('feature2 expected shape: (N, {}, {}, {})'.format(512, 480//16, 640//16))
print()

vgg16 = models.vgg16(weights=models.VGG16_Weights.IMAGENET1K_V1)
feature_extractor = FeatureExtraction(pretrained_model=vgg16)

dummy_input = {'rgb': torch.zeros((2,3,480,640))}
feature1, feature2 = feature_extractor(dummy_input)

print('feature1 shape:', feature1.shape)
print('feature2 shape:', feature2.shape)
```

```
feature1 expected shape: (N, 512, 60, 80)
feature2 expected shape: (N, 512, 30, 40)

feature1 shape: torch.Size([2, 512, 60, 80])
feature2 shape: torch.Size([2, 512, 30, 40])
```

Before moving on to the embedding and segmentation branch, take a moment to connect our `feature1` and `feature2` variables with the PoseCNN figure 2 shown earlier in the notebook. Understanding which arrows corresponds to `feature1` and `feature2` will make the remaining networks more comprehensible.

Implementing Segmentation Branch

Now that we have our feature extractor setup, we'll implement the instnace segmentation branch. This branch should fuse information from the feature extractor (`feature1` and `feature2`) according to the architecture diagram of PoseCNN. Specifically, the network will pass both outputs from the feature extractor through a 1x1 convolution+ReLU layer followed by interpolation and an element wise addition. Next these intermediate features are interpolated back to the input image size followed by a final 1x1 convolution+ReLU layer to predict a probability for each class or background at each pixel. Follow the instructions in the `SegmentationBranch` class of `pose_cnn.py` to implement this branch.

```
import torchvision.models as models
from utils import reset_seed
from pose_cnn import FeatureExtraction, SegmentationBranch

reset_seed(0)

vgg16 = models.vgg16(weights=models.VGG16_Weights.IMAGENET1K_V1)
feature_extractor = FeatureExtraction(pretrained_model=vgg16)
segmentation_branch = SegmentationBranch()

dummy_input = {'rgb': torch.zeros((2,3,480,640))}
feature1, feature2 = feature_extractor(dummy_input)
probability, segmentation, bbx = segmentation_branch(feature1, feature2)

print('probability expected shape: (B, 11, 480, 640)')
print('segmentation expected shape: (B, 480, 640)')
print('bbx expected shape: (N, 6) (where N is the number of rois extracted from
the predicted segmentation)')
print()
print('probability shape:', probability.shape)
print('segmentation shape:', segmentation.shape)
print('bbx shape:', bbx.shape)
```

```
probability expected shape: (B, 11, 480, 640)
segmentation expected shape: (B, 480, 640)
bbx expected shape: (N, 6) (where N is the number of rois extracted from the
predicted segmentation)

probability shape: torch.Size([2, 11, 480, 640])
segmentation shape: torch.Size([2, 480, 640])
bbx shape: torch.Size([6, 6])
```

Adding Instance Segmentation Branch to PoseCNN

Now that we have our instance segmentation branch implemented, we can begin building up our `PoseCNN` class. In `pose_cnn.py` use the given `FeatureExtraction` class and your implemented `SegmentationBranch` to initialize the extraction and segmentation portion of PoseCNN. Next, implement the training and testing time forward pass of `PoseCNN` to perform only instance segmentation. We will come back and add rotation+translation branches once we are confident in our instance segmentation.

Training PoseCNN to Perform Instance Segmentation

Once you've added code to initialize and perform the forward pass of PoseCNN for feature extraction and instance segmentation, we can attempt to train this part of PoseCNN by itself. The code in the following cell will initialize a PoseCNN model and begin training it on instance segmentation only. You should expect to see your training loss decrease to ~0.1 after training for 2 epochs.

```
import time
from torch.utils.data import DataLoader
import torchvision.models as models

from utils import reset_seed
from pose_cnn import PoseCNN
from tqdm import tqdm

reset_seed(0)

vgg16 = models.vgg16(weights=models.VGG16_Weights.IMAGENET1K_V1).to(DEVICE)
posecnn_model = PoseCNN(pretrained_backbone = vgg16,
                        models_pcd =
torch.tensor(train_dataset.models_pcd).to(DEVICE, dtype=torch.float32),
                        cam_intrinsic = train_dataset.cam_intrinsic).to(DEVICE)

dataloader = DataLoader(dataset=train_dataset, batch_size=BATCH_SIZE)
optimizer = torch.optim.Adam(posecnn_model.parameters(), lr=0.001,
                                betas=(0.9, 0.999))

posecnn_model.train()

loss_history = []
log_period = 5
_iter = 0

st_time = time.time()
for epoch in range(3):
    train_loss = []
    for batch in tqdm(dataloader):
        for item in batch:
            batch[item] = batch[item].to(DEVICE)
        loss_dict = posecnn_model(batch)
        optimizer.zero_grad()
        total_loss = loss_dict["loss_segmentation"]
        total_loss.backward()
        optimizer.step()
```

```
        train_loss.append(total_loss.item())

        if _iter % log_period == 0:
            loss_history.append(total_loss.item())
        _iter += 1

    print('Time {0}'.format(time.strftime("%Hh %Mm %Ss", time.gmtime(time.time()
- st_time)) + \
                                    ', ' + 'Epoch %02d' % epoch + ', ' + 'Training
finished' + f' , with mean training loss {np.array(train_loss).mean()}'))

plt.title("Training loss history")
plt.xlabel(f"Iteration (x {log_period})")
plt.ylabel("Loss")
plt.plot(loss_history)
plt.show()
```

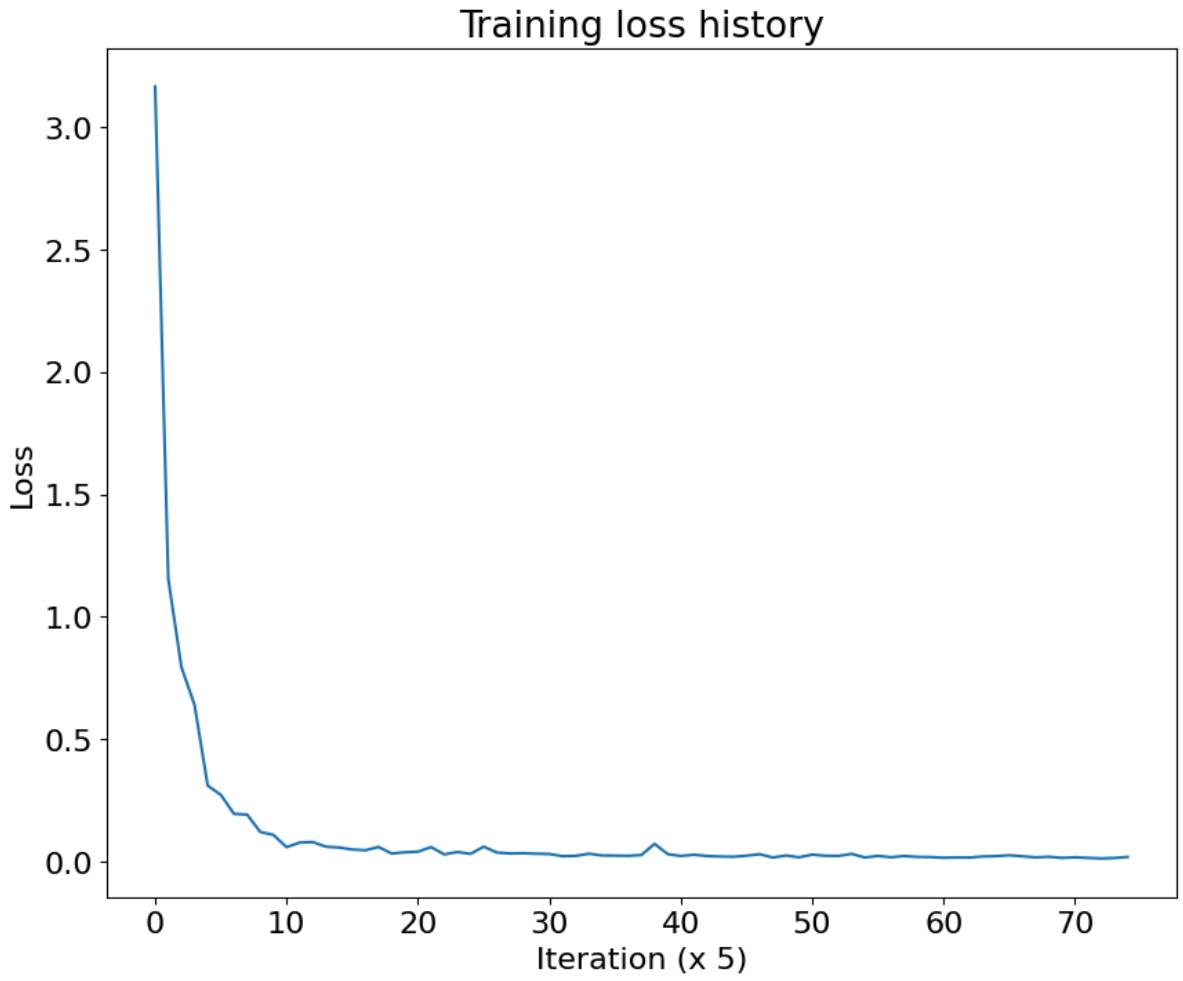
```
  0%|          | 0/125 [00:00<?, ?
it/s]/home/yuhuang/disk0/gglearn/db_robot_manip/Embodied-
AI/p3_pose/pose_cnn.py:403: UserWarning: The torch.cuda.*DtypeTensor constructors
are no longer recommended. It's best to use methods such as torch.tensor(data,
dtype=*, device='cuda') to create tensors. (Triggered internally at
/opt/conda/conda-
bld/pytorch_1724789259345/work/torch/csrc/tensor/python_tensor.cpp:78.)
  gt_bbx = gt_bbx.to(torch.cuda.FloatTensor())
100%|██████████| 125/125 [01:26<00:00,  1.45it/s]
```

```
Time 00h 01m 26s, Epoch 00, Training finished , with mean training loss
0.24991931423544883
```

```
100%|██████████| 125/125 [01:27<00:00,  1.43it/s]
```

```
Time 00h 02m 53s, Epoch 01, Training finished , with mean training loss
0.02915502329170704
```

```
100%|██████████| 125/125 [01:27<00:00,  1.43it/s]

Time 00h 04m 21s, Epoch 02, Training finished , with mean training loss
0.020453101195394993
```

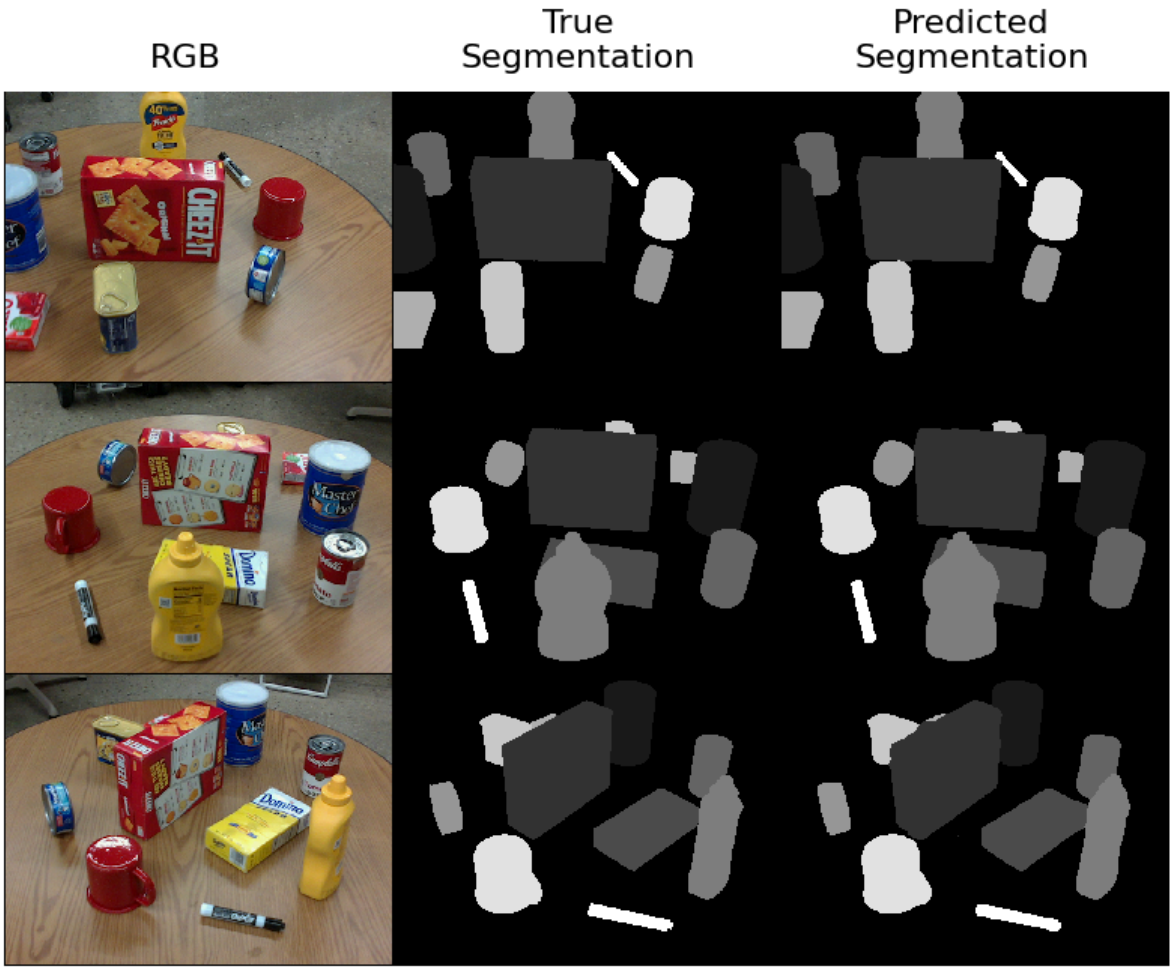


Inference for Instance Segmentation

Now that we have our segmentation network trained, we can qualitatively evaluate the segmentation results. The following notebook cell will visualize output segmentations on a sample from the validation set.

```
from torchvision.utils import make_grid

reset_seed(0)

num_samples = 3
posecnn_model.eval()

plt.text(300, -40, 'RGB', ha="center")
plt.text(950, -40, 'True\nSegmentation', ha="center")
plt.text(1600, -40, 'Predicted\nSegmentation', ha="center")

samples = []
for sample_i in range(num_samples):
    sample_idx = random.randint(0,len(val_dataset)-1)
    sample = val_dataset[sample_idx]

    rgb = torch.tensor(sample['rgb'][None, :]).to(DEVICE)
    _, prediction = posecnn_model({'rgb': rgb})
    prediction = prediction.cpu().numpy().astype(np.float64)
    prediction /= prediction.max()
    prediction = (np.tile(prediction, (3, 1, 1)) * 255).astype(np.uint8)
```

```
    rgb = (sample['rgb'].transpose(1, 2, 0) * 255).astype(np.uint8)
    depth = ((np.tile(sample['depth'], (3, 1, 1)) / sample['depth'].max()) *
255).astype(np.uint8)
    segmentation =
(sample['label']*np.arange(11).reshape((11,1,1))).sum(0,keepdims=True).astype(np.
float64)
    segmentation /= segmentation.max()
    segmentation = (np.tile(segmentation, (3, 1, 1)) * 255).astype(np.uint8)

    samples.append(torch.tensor(rgb.transpose(2, 0, 1)))
    samples.append(torch.tensor(segmentation))
    samples.append(torch.tensor(prediction))

img = make_grid(samples, nrow=3).permute(1, 2, 0)

plt.axis('off')
plt.imshow(img)
plt.show()
```



Before moving on, visually inspect the segmentation results above to ensure your forward functions and loss calculations are setup correctly.

Implementing the Translation Branch

Now that we have our feature extractor and instance segmentation working, we'll implement the rotation and translation branches of PoseCNN. We suggest starting with the translation branch, which follows a similar embedding structure as was used for instance segmentation. Based on the architecture described in the paper, implement your translation network in the `TranslationBranch` class of `pose_cnn.py`.

Once you have implemented `TranslationBranch`, the following notebook cell can be used to verify the shape of your output.

```
import torchvision.models as models
from utils import reset_seed
from pose_cnn import FeatureExtraction, TranslationBranch

reset_seed(0)

vgg16 = models.vgg16(weights=models.VGG16_Weights.IMAGENET1K_V1)
feature_extractor = FeatureExtraction(pretrained_model=vgg16)
translation_branch = TranslationBranch()

dummy_input = {'rgb': torch.zeros((2,3,480,640))}
feature1, feature2 = feature_extractor(dummy_input)
translation = translation_branch(feature1, feature2)

print('translation expected shape: (2, 30, 480, 640)')
print()
print('translation shape:', translation.shape)
```

```
translation expected shape: (2, 30, 480, 640)

translation shape: torch.Size([2, 30, 480, 640])
```

Implementing the Rotation Branch

Now you can implement the final module of PoseCNN: the rotation branch. This portion of PoseCNN will be reminiscient of fasterRCNN in that we will predict a quaternion for each possible class at each region of interest detected by our preceeding segmentation branch. To do this, we'll need to use ROIPooling for feature extraction. Since you implemented ROIPooling in the last project, feel free to use the ROIPool implemented by `torchvision.ops` for this project.

Once you have implemented `RotationBranch`, the following notebook cell can be used to verify the shape of your output.

```
import torchvision.models as models
from utils import reset_seed
from pose_cnn import FeatureExtraction, RotationBranch

reset_seed(0)

vgg16 = models.vgg16(weights=models.VGG16_Weights.IMAGENET1K_V1)
feature_extractor = FeatureExtraction(pretrained_model=vgg16)
rotation_branch = RotationBranch()
```

```
dummy_input = {'rgb': torch.zeros((2,3,480,640))}
dummy_bbx = torch.zeros((10,5)) # N = 10
feature1, feature2 = feature_extractor(dummy_input)
quaternion = rotation_branch(feature1, feature2, dummy_bbx)

print('quaternion expected shape: (N, 40)')
print()
print('quaternion shape:', quaternion.shape)
```

```
quaternion expected shape: (N, 40)

quaternion shape: torch.Size([10, 40])
```

Hough Voting Layer

One important piece of the PoseCNN architecture for inference time that we haven't implemented yet is a Hough voting layer. As described in the text, and illustrated below, a Hough voting layer is used during inference time to extract a single centroid prediction from the translation maps produced by `TranslationBranch` and the segments produced by `SegmentationBranch`. Due to the timing constraints of our semester, we have implemented this Hough voting layer for you in `P3_helper.py` as the `HoughVoting` function. We recommend you use our provided `HoughVoting` function after reading through its implementation to understand how it works

We also recommend reading through other given helper functions in `p3_helper.py` and the class definition of `PoseCNN` in `pose_cnn.py`. **Some of these utilities will be helpful for completing your PoseCNN implementation.**

Putting it all together: PoseCNN

We now have all the modules needed to make up our PoseCNN architecture. In the `PoseCNN` class of `pose_cnn.py`, add the translation and rotation branches to the initialization and forward functions. During training, your PoseCNN model should output a `loss_dict` variable with loss values for segmentation, translation and rotation branches stored respectively with keys of `"loss_segmentation"`, `"loss_centermap"`, and `"loss_R"`. The segmentation loss should be calculated using `p3_helper.loss_cross_entropy`, the centroid loss should be calculated using l1Loss, and the rotation loss should be calculated using the provided helper in `p3_helper.loss_Rotation`. During inference, your model should output a dictionary of predicted poses (i.e. see `PoseCNN.generate_pose` for a formatting utility) in `output_dict` and the predicted segmentation map (post processed probabilities) in `segmentation`.

After this, you will be ready to train your PoseCNN model with all three losses:

```
import os
import time
import torch
from torch.utils.data import DataLoader
import torchvision.models as models

import utils
from pose_cnn import PoseCNN
```

```
utils.reset_seed(0)

dataloader = DataLoader(dataset=train_dataset, batch_size=BATCH_SIZE)

vgg16 = models.vgg16(weights=models.VGG16_Weights.IMAGENET1K_V1)
posecnn_model = PoseCNN(pretrained_backbone = vgg16,
                models_pcd = torch.tensor(train_dataset.models_pcd).to(DEVICE,
dtype=torch.float32),
                cam_intrinsic = train_dataset.cam_intrinsic).to(DEVICE)
posecnn_model.train()

optimizer = torch.optim.Adam(posecnn_model.parameters(), lr=0.001,
                             betas=(0.9, 0.999))


loss_history = []
log_period = 5
_iter = 0


st_time = time.time()
for epoch in range(10):
    train_loss = []
    dataloader.dataset.dataset_type = 'train'
    for batch in dataloader:
        for item in batch:
            batch[item] = batch[item].to(DEVICE)
        loss_dict = posecnn_model(batch)
        optimizer.zero_grad()
        total_loss = 0
        for loss in loss_dict:
            total_loss += loss_dict[loss]
        total_loss.backward()
        optimizer.step()
        train_loss.append(total_loss.item())

        if _iter % log_period == 0:
            loss_str = f"[Iter {_iter}][loss: {total_loss:.3f}]"
            for key, value in loss_dict.items():
                loss_str += f"[{key}: {value:.3f}]"

            print(loss_str)
            loss_history.append(total_loss.item())
        _iter += 1

    print('Time {0}'.format(time.strftime("%Hh %Mm %Ss", time.gmtime(time.time()
- st_time)) + \
                                    ', ' + 'Epoch %02d' % epoch + ', ' + 'Training
finished' + f' , with mean training loss {np.array(train_loss).mean()}'))

torch.save(posecnn_model.state_dict(), os.path.join(GOOGLE_DRIVE_PATH,
"posecnn_model.pth"))

plt.title("Training loss history")
plt.xlabel(f"Iteration (x {log_period})")
```

```
plt.ylabel("Loss")
plt.plot(loss_history)
plt.show()
```

```
[Iter 0][loss: 4.439][loss_segmentation: 3.166][loss_centermap: 1.273][loss_R:
0.000]
[Iter 5][loss: 1.691][loss_segmentation: 0.921][loss_centermap: 0.770][loss_R:
0.000]
[Iter 10][loss: 1.453][loss_segmentation: 0.852][loss_centermap: 0.601][loss_R:
0.000]
[Iter 15][loss: 1.381][loss_segmentation: 0.698][loss_centermap: 0.573][loss_R:
0.110]
[Iter 20][loss: 0.952][loss_segmentation: 0.434][loss_centermap: 0.517][loss_R:
0.000]
[Iter 25][loss: 1.037][loss_segmentation: 0.440][loss_centermap: 0.524][loss_R:
0.073]
[Iter 30][loss: 1.040][loss_segmentation: 0.372][loss_centermap: 0.531][loss_R:
0.137]
[Iter 35][loss: 0.801][loss_segmentation: 0.234][loss_centermap: 0.486][loss_R:
0.081]
[Iter 40][loss: 0.725][loss_segmentation: 0.206][loss_centermap: 0.445][loss_R:
0.074]
[Iter 45][loss: 0.688][loss_segmentation: 0.146][loss_centermap: 0.475][loss_R:
0.067]
[Iter 50][loss: 0.576][loss_segmentation: 0.091][loss_centermap: 0.433][loss_R:
0.052]
[Iter 55][loss: 0.573][loss_segmentation: 0.103][loss_centermap: 0.424][loss_R:
0.046]
[Iter 60][loss: 0.603][loss_segmentation: 0.101][loss_centermap: 0.443][loss_R:
0.059]
[Iter 65][loss: 0.556][loss_segmentation: 0.094][loss_centermap: 0.409][loss_R:
0.054]
[Iter 70][loss: 0.549][loss_segmentation: 0.083][loss_centermap: 0.404][loss_R:
0.062]
[Iter 75][loss: 0.519][loss_segmentation: 0.072][loss_centermap: 0.384][loss_R:
0.063]
[Iter 80][loss: 0.507][loss_segmentation: 0.058][loss_centermap: 0.389][loss_R:
0.060]
[Iter 85][loss: 0.541][loss_segmentation: 0.089][loss_centermap: 0.403][loss_R:
0.049]
[Iter 90][loss: 0.422][loss_segmentation: 0.047][loss_centermap: 0.331][loss_R:
0.044]
[Iter 95][loss: 0.441][loss_segmentation: 0.051][loss_centermap: 0.343][loss_R:
0.047]
[Iter 100][loss: 0.451][loss_segmentation: 0.055][loss_centermap: 0.343][loss_R:
0.053]
[Iter 105][loss: 0.473][loss_segmentation: 0.062][loss_centermap: 0.345][loss_R:
0.065]
[Iter 110][loss: 0.410][loss_segmentation: 0.033][loss_centermap: 0.342][loss_R:
0.036]
[Iter 115][loss: 0.388][loss_segmentation: 0.047][loss_centermap: 0.300][loss_R:
0.041]
[Iter 120][loss: 0.392][loss_segmentation: 0.036][loss_centermap: 0.308][loss_R:
0.048]
```

```
Time 00h 01m 31s, Epoch 00, Training finished , with mean training loss
0.7743997557163238
[Iter 125][loss: 0.436][loss_segmentation: 0.066][loss_centermap: 0.319][loss_R:
0.051]
[Iter 130][loss: 0.380][loss_segmentation: 0.037][loss_centermap: 0.293][loss_R:
0.050]
[Iter 135][loss: 0.395][loss_segmentation: 0.039][loss_centermap: 0.302][loss_R:
0.055]
[Iter 140][loss: 0.376][loss_segmentation: 0.040][loss_centermap: 0.285][loss_R:
0.051]
[Iter 145][loss: 0.345][loss_segmentation: 0.038][loss_centermap: 0.271][loss_R:
0.037]
[Iter 150][loss: 0.389][loss_segmentation: 0.038][loss_centermap: 0.292][loss_R:
0.059]
[Iter 155][loss: 0.381][loss_segmentation: 0.033][loss_centermap: 0.295][loss_R:
0.054]
[Iter 160][loss: 0.342][loss_segmentation: 0.032][loss_centermap: 0.265][loss_R:
0.045]
[Iter 165][loss: 0.347][loss_segmentation: 0.042][loss_centermap: 0.262][loss_R:
0.042]
[Iter 170][loss: 0.385][loss_segmentation: 0.037][loss_centermap: 0.298][loss_R:
0.050]
[Iter 175][loss: 0.313][loss_segmentation: 0.029][loss_centermap: 0.246][loss_R:
0.038]
[Iter 180][loss: 0.312][loss_segmentation: 0.030][loss_centermap: 0.248][loss_R:
0.033]
[Iter 185][loss: 0.345][loss_segmentation: 0.033][loss_centermap: 0.258][loss_R:
0.054]
[Iter 190][loss: 0.410][loss_segmentation: 0.077][loss_centermap: 0.279][loss_R:
0.054]
[Iter 195][loss: 0.363][loss_segmentation: 0.033][loss_centermap: 0.266][loss_R:
0.063]
[Iter 200][loss: 0.329][loss_segmentation: 0.031][loss_centermap: 0.236][loss_R:
0.061]
[Iter 205][loss: 0.301][loss_segmentation: 0.027][loss_centermap: 0.233][loss_R:
0.041]
[Iter 210][loss: 0.300][loss_segmentation: 0.031][loss_centermap: 0.223][loss_R:
0.045]
[Iter 215][loss: 0.237][loss_segmentation: 0.026][loss_centermap: 0.185][loss_R:
0.026]
[Iter 220][loss: 0.268][loss_segmentation: 0.026][loss_centermap: 0.209][loss_R:
0.032]
[Iter 225][loss: 0.283][loss_segmentation: 0.032][loss_centermap: 0.215][loss_R:
0.036]
[Iter 230][loss: 0.321][loss_segmentation: 0.037][loss_centermap: 0.229][loss_R:
0.055]
[Iter 235][loss: 0.208][loss_segmentation: 0.022][loss_centermap: 0.166][loss_R:
0.020]
[Iter 240][loss: 0.242][loss_segmentation: 0.031][loss_centermap: 0.189][loss_R:
0.022]
[Iter 245][loss: 0.256][loss_segmentation: 0.026][loss_centermap: 0.191][loss_R:
0.039]
Time 00h 03m 06s, Epoch 01, Training finished , with mean training loss
0.3328759038448334
[Iter 250][loss: 0.247][loss_segmentation: 0.030][loss_centermap: 0.181][loss_R:
0.036]
```

```
[Iter 255][loss: 0.232][loss_segmentation: 0.025][loss_centermap: 0.170][loss_R:
0.037]
[Iter 260][loss: 0.236][loss_segmentation: 0.027][loss_centermap: 0.177][loss_R:
0.032]
[Iter 265][loss: 0.268][loss_segmentation: 0.035][loss_centermap: 0.198][loss_R:
0.035]
[Iter 270][loss: 0.201][loss_segmentation: 0.020][loss_centermap: 0.161][loss_R:
0.021]
[Iter 275][loss: 0.271][loss_segmentation: 0.028][loss_centermap: 0.204][loss_R:
0.039]
[Iter 280][loss: 0.229][loss_segmentation: 0.025][loss_centermap: 0.174][loss_R:
0.030]
[Iter 285][loss: 0.229][loss_segmentation: 0.030][loss_centermap: 0.174][loss_R:
0.024]
[Iter 290][loss: 0.193][loss_segmentation: 0.024][loss_centermap: 0.154][loss_R:
0.016]
[Iter 295][loss: 0.238][loss_segmentation: 0.023][loss_centermap: 0.189][loss_R:
0.027]
[Iter 300][loss: 0.190][loss_segmentation: 0.021][loss_centermap: 0.155][loss_R:
0.014]
[Iter 305][loss: 0.177][loss_segmentation: 0.021][loss_centermap: 0.143][loss_R:
0.013]
[Iter 310][loss: 0.208][loss_segmentation: 0.024][loss_centermap: 0.165][loss_R:
0.019]
[Iter 315][loss: 0.200][loss_segmentation: 0.025][loss_centermap: 0.153][loss_R:
0.022]
[Iter 320][loss: 0.216][loss_segmentation: 0.027][loss_centermap: 0.164][loss_R:
0.025]
[Iter 325][loss: 0.217][loss_segmentation: 0.028][loss_centermap: 0.159][loss_R:
0.030]
[Iter 330][loss: 0.199][loss_segmentation: 0.027][loss_centermap: 0.151][loss_R:
0.020]
[Iter 335][loss: 0.176][loss_segmentation: 0.022][loss_centermap: 0.133][loss_R:
0.021]
[Iter 340][loss: 0.173][loss_segmentation: 0.022][loss_centermap: 0.140][loss_R:
0.011]
[Iter 345][loss: 0.192][loss_segmentation: 0.020][loss_centermap: 0.157][loss_R:
0.015]
[Iter 350][loss: 0.162][loss_segmentation: 0.023][loss_centermap: 0.119][loss_R:
0.020]
[Iter 355][loss: 0.202][loss_segmentation: 0.020][loss_centermap: 0.154][loss_R:
0.028]
[Iter 360][loss: 0.143][loss_segmentation: 0.019][loss_centermap: 0.113][loss_R:
0.011]
[Iter 365][loss: 0.151][loss_segmentation: 0.021][loss_centermap: 0.115][loss_R:
0.014]
[Iter 370][loss: 0.207][loss_segmentation: 0.025][loss_centermap: 0.146][loss_R:
0.036]
Time 00h 04m 42s, Epoch 02, Training finished , with mean training loss
0.20974727034568785
[Iter 375][loss: 0.158][loss_segmentation: 0.021][loss_centermap: 0.119][loss_R:
0.018]
[Iter 380][loss: 0.174][loss_segmentation: 0.023][loss_centermap: 0.123][loss_R:
0.028]
[Iter 385][loss: 0.177][loss_segmentation: 0.025][loss_centermap: 0.133][loss_R:
0.020]
```

```
[Iter 390][loss: 0.243][loss_segmentation: 0.044][loss_centermap: 0.143][loss_R:
0.056]
[Iter 395][loss: 0.157][loss_segmentation: 0.019][loss_centermap: 0.125][loss_R:
0.014]
[Iter 400][loss: 0.201][loss_segmentation: 0.025][loss_centermap: 0.153][loss_R:
0.023]
[Iter 405][loss: 0.194][loss_segmentation: 0.024][loss_centermap: 0.138][loss_R:
0.032]
[Iter 410][loss: 0.179][loss_segmentation: 0.021][loss_centermap: 0.138][loss_R:
0.020]
[Iter 415][loss: 0.223][loss_segmentation: 0.040][loss_centermap: 0.152][loss_R:
0.032]
[Iter 420][loss: 0.199][loss_segmentation: 0.021][loss_centermap: 0.152][loss_R:
0.025]
[Iter 425][loss: 0.156][loss_segmentation: 0.018][loss_centermap: 0.125][loss_R:
0.014]
[Iter 430][loss: 0.145][loss_segmentation: 0.017][loss_centermap: 0.111][loss_R:
0.016]
[Iter 435][loss: 0.175][loss_segmentation: 0.021][loss_centermap: 0.136][loss_R:
0.018]
[Iter 440][loss: 0.157][loss_segmentation: 0.020][loss_centermap: 0.117][loss_R:
0.019]
[Iter 445][loss: 0.169][loss_segmentation: 0.018][loss_centermap: 0.130][loss_R:
0.020]
[Iter 450][loss: 0.158][loss_segmentation: 0.022][loss_centermap: 0.111][loss_R:
0.026]
[Iter 455][loss: 0.144][loss_segmentation: 0.017][loss_centermap: 0.106][loss_R:
0.020]
[Iter 460][loss: 0.142][loss_segmentation: 0.019][loss_centermap: 0.104][loss_R:
0.019]
[Iter 465][loss: 0.126][loss_segmentation: 0.016][loss_centermap: 0.100][loss_R:
0.010]
[Iter 470][loss: 0.154][loss_segmentation: 0.017][loss_centermap: 0.118][loss_R:
0.020]
[Iter 475][loss: 0.129][loss_segmentation: 0.017][loss_centermap: 0.093][loss_R:
0.019]
[Iter 480][loss: 0.188][loss_segmentation: 0.022][loss_centermap: 0.141][loss_R:
0.025]
[Iter 485][loss: 0.111][loss_segmentation: 0.015][loss_centermap: 0.086][loss_R:
0.009]
[Iter 490][loss: 0.124][loss_segmentation: 0.018][loss_centermap: 0.094][loss_R:
0.011]
[Iter 495][loss: 0.149][loss_segmentation: 0.018][loss_centermap: 0.108][loss_R:
0.023]
Time 00h 06m 18s, Epoch 03, Training finished , with mean training loss
0.16726659548282624
[Iter 500][loss: 0.143][loss_segmentation: 0.026][loss_centermap: 0.103][loss_R:
0.014]
[Iter 505][loss: 0.159][loss_segmentation: 0.024][loss_centermap: 0.109][loss_R:
0.026]
[Iter 510][loss: 0.134][loss_segmentation: 0.022][loss_centermap: 0.093][loss_R:
0.018]
[Iter 515][loss: 0.188][loss_segmentation: 0.035][loss_centermap: 0.127][loss_R:
0.026]
[Iter 520][loss: 0.131][loss_segmentation: 0.017][loss_centermap: 0.101][loss_R:
0.012]
```

```
[Iter 525][loss: 0.232][loss_segmentation: 0.032][loss_centermap: 0.150][loss_R:
0.051]
[Iter 530][loss: 0.154][loss_segmentation: 0.017][loss_centermap: 0.113][loss_R:
0.024]
[Iter 535][loss: 0.182][loss_segmentation: 0.033][loss_centermap: 0.119][loss_R:
0.030]
[Iter 540][loss: 0.142][loss_segmentation: 0.023][loss_centermap: 0.107][loss_R:
0.012]
[Iter 545][loss: 0.189][loss_segmentation: 0.018][loss_centermap: 0.148][loss_R:
0.023]
[Iter 550][loss: 0.136][loss_segmentation: 0.015][loss_centermap: 0.109][loss_R:
0.012]
[Iter 555][loss: 0.126][loss_segmentation: 0.015][loss_centermap: 0.096][loss_R:
0.014]
[Iter 560][loss: 0.136][loss_segmentation: 0.018][loss_centermap: 0.100][loss_R:
0.018]
[Iter 565][loss: 0.153][loss_segmentation: 0.027][loss_centermap: 0.108][loss_R:
0.018]
[Iter 570][loss: 0.149][loss_segmentation: 0.018][loss_centermap: 0.110][loss_R:
0.020]
[Iter 575][loss: 0.122][loss_segmentation: 0.015][loss_centermap: 0.085][loss_R:
0.022]
[Iter 580][loss: 0.136][loss_segmentation: 0.016][loss_centermap: 0.101][loss_R:
0.019]
[Iter 585][loss: 0.129][loss_segmentation: 0.018][loss_centermap: 0.092][loss_R:
0.020]
[Iter 590][loss: 0.125][loss_segmentation: 0.014][loss_centermap: 0.099][loss_R:
0.013]
[Iter 595][loss: 0.161][loss_segmentation: 0.019][loss_centermap: 0.127][loss_R:
0.015]
[Iter 600][loss: 0.124][loss_segmentation: 0.016][loss_centermap: 0.091][loss_R:
0.017]
[Iter 605][loss: 0.151][loss_segmentation: 0.014][loss_centermap: 0.116][loss_R:
0.021]
[Iter 610][loss: 0.095][loss_segmentation: 0.015][loss_centermap: 0.072][loss_R:
0.008]
[Iter 615][loss: 0.113][loss_segmentation: 0.017][loss_centermap: 0.086][loss_R:
0.009]
[Iter 620][loss: 0.131][loss_segmentation: 0.015][loss_centermap: 0.093][loss_R:
0.024]
Time 00h 07m 53s, Epoch 04, Training finished , with mean training loss
0.14230725741386413
[Iter 625][loss: 0.109][loss_segmentation: 0.018][loss_centermap: 0.077][loss_R:
0.014]
[Iter 630][loss: 0.102][loss_segmentation: 0.015][loss_centermap: 0.070][loss_R:
0.017]
[Iter 635][loss: 0.108][loss_segmentation: 0.017][loss_centermap: 0.074][loss_R:
0.017]
[Iter 640][loss: 0.113][loss_segmentation: 0.016][loss_centermap: 0.080][loss_R:
0.017]
[Iter 645][loss: 0.118][loss_segmentation: 0.015][loss_centermap: 0.092][loss_R:
0.011]
[Iter 650][loss: 0.156][loss_segmentation: 0.015][loss_centermap: 0.118][loss_R:
0.022]
[Iter 655][loss: 0.133][loss_segmentation: 0.014][loss_centermap: 0.098][loss_R:
0.021]
```

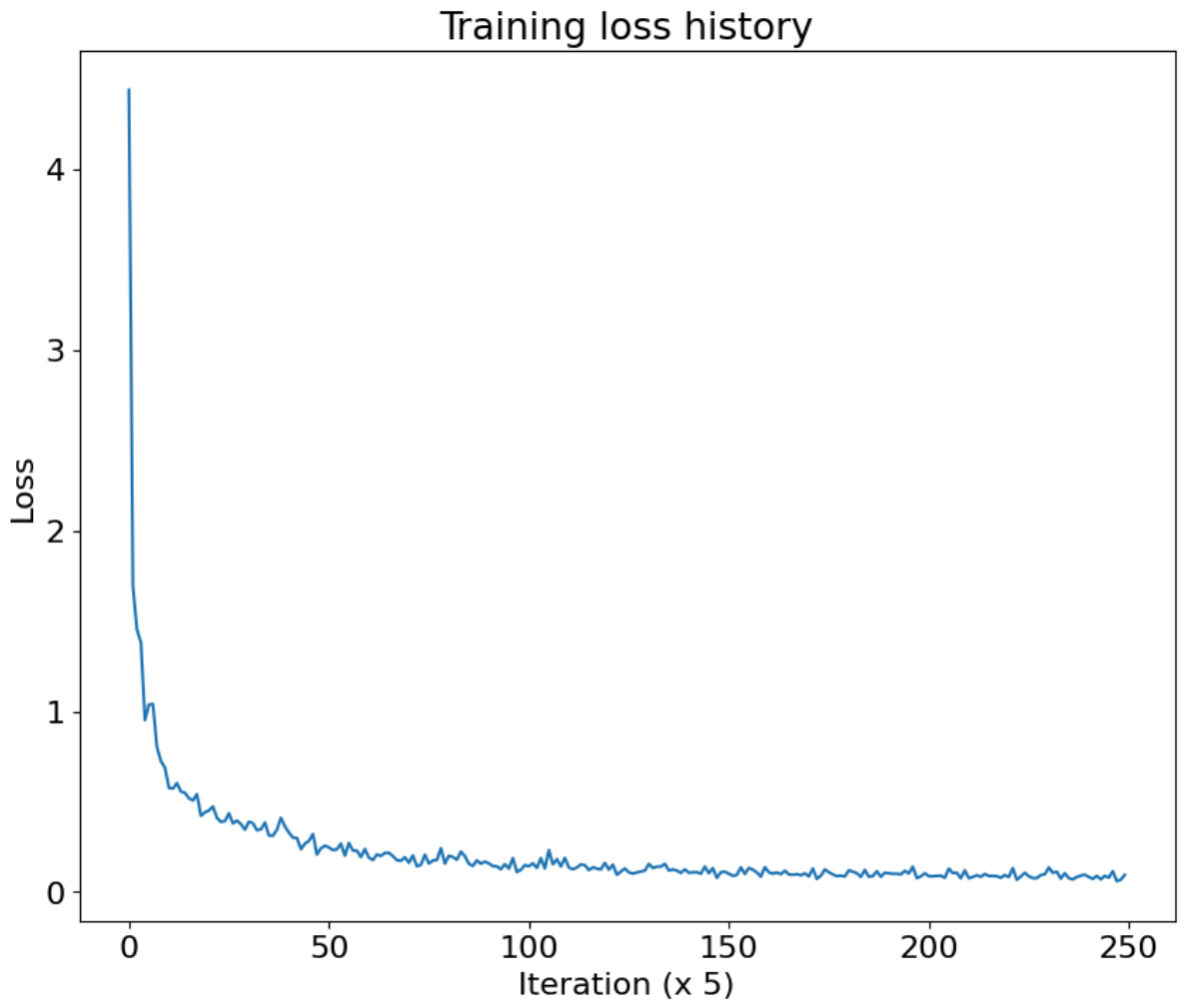
```
[Iter 660][loss: 0.140][loss_segmentation: 0.016][loss_centermap: 0.104][loss_R:
0.019]
[Iter 665][loss: 0.141][loss_segmentation: 0.024][loss_centermap: 0.110][loss_R:
0.007]
[Iter 670][loss: 0.156][loss_segmentation: 0.018][loss_centermap: 0.127][loss_R:
0.012]
[Iter 675][loss: 0.119][loss_segmentation: 0.015][loss_centermap: 0.095][loss_R:
0.010]
[Iter 680][loss: 0.125][loss_segmentation: 0.018][loss_centermap: 0.093][loss_R:
0.013]
[Iter 685][loss: 0.118][loss_segmentation: 0.013][loss_centermap: 0.088][loss_R:
0.017]
[Iter 690][loss: 0.106][loss_segmentation: 0.016][loss_centermap: 0.075][loss_R:
0.015]
[Iter 695][loss: 0.125][loss_segmentation: 0.016][loss_centermap: 0.087][loss_R:
0.023]
[Iter 700][loss: 0.107][loss_segmentation: 0.015][loss_centermap: 0.071][loss_R:
0.021]
[Iter 705][loss: 0.109][loss_segmentation: 0.013][loss_centermap: 0.079][loss_R:
0.017]
[Iter 710][loss: 0.110][loss_segmentation: 0.014][loss_centermap: 0.078][loss_R:
0.018]
[Iter 715][loss: 0.101][loss_segmentation: 0.012][loss_centermap: 0.080][loss_R:
0.009]
[Iter 720][loss: 0.141][loss_segmentation: 0.017][loss_centermap: 0.110][loss_R:
0.014]
[Iter 725][loss: 0.104][loss_segmentation: 0.015][loss_centermap: 0.074][loss_R:
0.015]
[Iter 730][loss: 0.133][loss_segmentation: 0.013][loss_centermap: 0.098][loss_R:
0.021]
[Iter 735][loss: 0.078][loss_segmentation: 0.011][loss_centermap: 0.059][loss_R:
0.007]
[Iter 740][loss: 0.110][loss_segmentation: 0.016][loss_centermap: 0.075][loss_R:
0.019]
[Iter 745][loss: 0.113][loss_segmentation: 0.013][loss_centermap: 0.082][loss_R:
0.018]
Time 00h 09m 30s, Epoch 05, Training finished , with mean training loss
0.12345130079984665
[Iter 750][loss: 0.101][loss_segmentation: 0.015][loss_centermap: 0.073][loss_R:
0.013]
[Iter 755][loss: 0.090][loss_segmentation: 0.014][loss_centermap: 0.062][loss_R:
0.014]
[Iter 760][loss: 0.095][loss_segmentation: 0.015][loss_centermap: 0.064][loss_R:
0.016]
[Iter 765][loss: 0.136][loss_segmentation: 0.025][loss_centermap: 0.080][loss_R:
0.031]
[Iter 770][loss: 0.100][loss_segmentation: 0.013][loss_centermap: 0.078][loss_R:
0.010]
[Iter 775][loss: 0.133][loss_segmentation: 0.014][loss_centermap: 0.094][loss_R:
0.025]
[Iter 780][loss: 0.124][loss_segmentation: 0.015][loss_centermap: 0.087][loss_R:
0.022]
[Iter 785][loss: 0.109][loss_segmentation: 0.014][loss_centermap: 0.078][loss_R:
0.017]
[Iter 790][loss: 0.086][loss_segmentation: 0.014][loss_centermap: 0.066][loss_R:
0.006]
```

```
[Iter 795][loss: 0.137][loss_segmentation: 0.015][loss_centermap: 0.109][loss_R:
0.013]
[Iter 800][loss: 0.109][loss_segmentation: 0.014][loss_centermap: 0.085][loss_R:
0.010]
[Iter 805][loss: 0.102][loss_segmentation: 0.014][loss_centermap: 0.076][loss_R:
0.011]
[Iter 810][loss: 0.108][loss_segmentation: 0.013][loss_centermap: 0.076][loss_R:
0.019]
[Iter 815][loss: 0.099][loss_segmentation: 0.017][loss_centermap: 0.067][loss_R:
0.015]
[Iter 820][loss: 0.118][loss_segmentation: 0.016][loss_centermap: 0.081][loss_R:
0.020]
[Iter 825][loss: 0.098][loss_segmentation: 0.014][loss_centermap: 0.063][loss_R:
0.021]
[Iter 830][loss: 0.095][loss_segmentation: 0.013][loss_centermap: 0.066][loss_R:
0.016]
[Iter 835][loss: 0.099][loss_segmentation: 0.014][loss_centermap: 0.068][loss_R:
0.018]
[Iter 840][loss: 0.092][loss_segmentation: 0.012][loss_centermap: 0.072][loss_R:
0.009]
[Iter 845][loss: 0.103][loss_segmentation: 0.012][loss_centermap: 0.077][loss_R:
0.014]
[Iter 850][loss: 0.087][loss_segmentation: 0.013][loss_centermap: 0.059][loss_R:
0.015]
[Iter 855][loss: 0.130][loss_segmentation: 0.013][loss_centermap: 0.096][loss_R:
0.022]
[Iter 860][loss: 0.073][loss_segmentation: 0.010][loss_centermap: 0.055][loss_R:
0.007]
[Iter 865][loss: 0.090][loss_segmentation: 0.012][loss_centermap: 0.069][loss_R:
0.010]
[Iter 870][loss: 0.125][loss_segmentation: 0.014][loss_centermap: 0.086][loss_R:
0.024]
Time 00h 11m 05s, Epoch 06, Training finished , with mean training loss
0.10854929304122925
[Iter 875][loss: 0.110][loss_segmentation: 0.020][loss_centermap: 0.078][loss_R:
0.012]
[Iter 880][loss: 0.097][loss_segmentation: 0.018][loss_centermap: 0.062][loss_R:
0.017]
[Iter 885][loss: 0.088][loss_segmentation: 0.013][loss_centermap: 0.059][loss_R:
0.016]
[Iter 890][loss: 0.091][loss_segmentation: 0.014][loss_centermap: 0.062][loss_R:
0.015]
[Iter 895][loss: 0.086][loss_segmentation: 0.011][loss_centermap: 0.066][loss_R:
0.009]
[Iter 900][loss: 0.120][loss_segmentation: 0.011][loss_centermap: 0.083][loss_R:
0.026]
[Iter 905][loss: 0.113][loss_segmentation: 0.013][loss_centermap: 0.080][loss_R:
0.020]
[Iter 910][loss: 0.104][loss_segmentation: 0.012][loss_centermap: 0.075][loss_R:
0.017]
[Iter 915][loss: 0.084][loss_segmentation: 0.012][loss_centermap: 0.065][loss_R:
0.007]
[Iter 920][loss: 0.122][loss_segmentation: 0.013][loss_centermap: 0.097][loss_R:
0.012]
[Iter 925][loss: 0.085][loss_segmentation: 0.012][loss_centermap: 0.065][loss_R:
0.008]
```

```
[Iter 930][loss: 0.088][loss_segmentation: 0.012][loss_centermap: 0.066][loss_R:
0.010]
[Iter 935][loss: 0.116][loss_segmentation: 0.014][loss_centermap: 0.084][loss_R:
0.017]
[Iter 940][loss: 0.085][loss_segmentation: 0.013][loss_centermap: 0.058][loss_R:
0.014]
[Iter 945][loss: 0.107][loss_segmentation: 0.013][loss_centermap: 0.074][loss_R:
0.020]
[Iter 950][loss: 0.103][loss_segmentation: 0.016][loss_centermap: 0.067][loss_R:
0.021]
[Iter 955][loss: 0.101][loss_segmentation: 0.015][loss_centermap: 0.069][loss_R:
0.017]
[Iter 960][loss: 0.101][loss_segmentation: 0.018][loss_centermap: 0.068][loss_R:
0.015]
[Iter 965][loss: 0.098][loss_segmentation: 0.012][loss_centermap: 0.078][loss_R:
0.008]
[Iter 970][loss: 0.117][loss_segmentation: 0.013][loss_centermap: 0.088][loss_R:
0.016]
[Iter 975][loss: 0.103][loss_segmentation: 0.017][loss_centermap: 0.072][loss_R:
0.015]
[Iter 980][loss: 0.140][loss_segmentation: 0.015][loss_centermap: 0.103][loss_R:
0.021]
[Iter 985][loss: 0.077][loss_segmentation: 0.013][loss_centermap: 0.057][loss_R:
0.006]
[Iter 990][loss: 0.087][loss_segmentation: 0.014][loss_centermap: 0.064][loss_R:
0.009]
[Iter 995][loss: 0.103][loss_segmentation: 0.012][loss_centermap: 0.075][loss_R:
0.016]
Time 00h 12m 40s, Epoch 07, Training finished , with mean training loss
0.10406867879629135
[Iter 1000][loss: 0.088][loss_segmentation: 0.015][loss_centermap: 0.061][loss_R:
0.012]
[Iter 1005][loss: 0.087][loss_segmentation: 0.015][loss_centermap: 0.057][loss_R:
0.016]
[Iter 1010][loss: 0.090][loss_segmentation: 0.016][loss_centermap: 0.058][loss_R:
0.016]
[Iter 1015][loss: 0.090][loss_segmentation: 0.014][loss_centermap: 0.062][loss_R:
0.015]
[Iter 1020][loss: 0.079][loss_segmentation: 0.012][loss_centermap: 0.059][loss_R:
0.009]
[Iter 1025][loss: 0.130][loss_segmentation: 0.016][loss_centermap: 0.095][loss_R:
0.019]
[Iter 1030][loss: 0.105][loss_segmentation: 0.013][loss_centermap: 0.075][loss_R:
0.018]
[Iter 1035][loss: 0.107][loss_segmentation: 0.015][loss_centermap: 0.076][loss_R:
0.016]
[Iter 1040][loss: 0.076][loss_segmentation: 0.011][loss_centermap: 0.059][loss_R:
0.006]
[Iter 1045][loss: 0.120][loss_segmentation: 0.011][loss_centermap: 0.097][loss_R:
0.012]
[Iter 1050][loss: 0.076][loss_segmentation: 0.010][loss_centermap: 0.060][loss_R:
0.006]
[Iter 1055][loss: 0.083][loss_segmentation: 0.013][loss_centermap: 0.059][loss_R:
0.011]
[Iter 1060][loss: 0.093][loss_segmentation: 0.012][loss_centermap: 0.066][loss_R:
0.016]
```

```
[Iter 1065][loss: 0.085][loss_segmentation: 0.012][loss_centermap: 0.058][loss_R:
0.014]
[Iter 1070][loss: 0.100][loss_segmentation: 0.012][loss_centermap: 0.068][loss_R:
0.019]
[Iter 1075][loss: 0.089][loss_segmentation: 0.013][loss_centermap: 0.057][loss_R:
0.019]
[Iter 1080][loss: 0.089][loss_segmentation: 0.013][loss_centermap: 0.061][loss_R:
0.015]
[Iter 1085][loss: 0.088][loss_segmentation: 0.013][loss_centermap: 0.060][loss_R:
0.015]
[Iter 1090][loss: 0.079][loss_segmentation: 0.011][loss_centermap: 0.059][loss_R:
0.008]
[Iter 1095][loss: 0.094][loss_segmentation: 0.013][loss_centermap: 0.069][loss_R:
0.012]
[Iter 1100][loss: 0.083][loss_segmentation: 0.013][loss_centermap: 0.056][loss_R:
0.014]
[Iter 1105][loss: 0.133][loss_segmentation: 0.012][loss_centermap: 0.098][loss_R:
0.023]
[Iter 1110][loss: 0.068][loss_segmentation: 0.010][loss_centermap: 0.052][loss_R:
0.006]
[Iter 1115][loss: 0.086][loss_segmentation: 0.012][loss_centermap: 0.064][loss_R:
0.010]
[Iter 1120][loss: 0.106][loss_segmentation: 0.012][loss_centermap: 0.079][loss_R:
0.016]
Time 00h 14m 16s, Epoch 08, Training finished , with mean training loss
0.09837104505300522
[Iter 1125][loss: 0.087][loss_segmentation: 0.014][loss_centermap: 0.060][loss_R:
0.013]
[Iter 1130][loss: 0.077][loss_segmentation: 0.012][loss_centermap: 0.051][loss_R:
0.013]
[Iter 1135][loss: 0.077][loss_segmentation: 0.013][loss_centermap: 0.049][loss_R:
0.015]
[Iter 1140][loss: 0.094][loss_segmentation: 0.017][loss_centermap: 0.061][loss_R:
0.016]
[Iter 1145][loss: 0.098][loss_segmentation: 0.018][loss_centermap: 0.071][loss_R:
0.009]
[Iter 1150][loss: 0.135][loss_segmentation: 0.013][loss_centermap: 0.090][loss_R:
0.032]
[Iter 1155][loss: 0.107][loss_segmentation: 0.013][loss_centermap: 0.076][loss_R:
0.018]
[Iter 1160][loss: 0.112][loss_segmentation: 0.014][loss_centermap: 0.075][loss_R:
0.023]
[Iter 1165][loss: 0.074][loss_segmentation: 0.014][loss_centermap: 0.055][loss_R:
0.006]
[Iter 1170][loss: 0.104][loss_segmentation: 0.011][loss_centermap: 0.081][loss_R:
0.012]
[Iter 1175][loss: 0.077][loss_segmentation: 0.012][loss_centermap: 0.057][loss_R:
0.008]
[Iter 1180][loss: 0.070][loss_segmentation: 0.010][loss_centermap: 0.050][loss_R:
0.010]
[Iter 1185][loss: 0.083][loss_segmentation: 0.012][loss_centermap: 0.055][loss_R:
0.016]
[Iter 1190][loss: 0.091][loss_segmentation: 0.016][loss_centermap: 0.061][loss_R:
0.014]
[Iter 1195][loss: 0.096][loss_segmentation: 0.012][loss_centermap: 0.063][loss_R:
0.021]
```

```
[Iter 1200][loss: 0.083][loss_segmentation: 0.013][loss_centermap: 0.051][loss_R:
0.019]
[Iter 1205][loss: 0.073][loss_segmentation: 0.011][loss_centermap: 0.047][loss_R:
0.015]
[Iter 1210][loss: 0.089][loss_segmentation: 0.015][loss_centermap: 0.056][loss_R:
0.019]
[Iter 1215][loss: 0.070][loss_segmentation: 0.010][loss_centermap: 0.053][loss_R:
0.008]
[Iter 1220][loss: 0.089][loss_segmentation: 0.012][loss_centermap: 0.061][loss_R:
0.016]
[Iter 1225][loss: 0.080][loss_segmentation: 0.014][loss_centermap: 0.054][loss_R:
0.012]
[Iter 1230][loss: 0.115][loss_segmentation: 0.011][loss_centermap: 0.082][loss_R:
0.023]
[Iter 1235][loss: 0.060][loss_segmentation: 0.009][loss_centermap: 0.043][loss_R:
0.007]
[Iter 1240][loss: 0.068][loss_segmentation: 0.010][loss_centermap: 0.050][loss_R:
0.008]
[Iter 1245][loss: 0.095][loss_segmentation: 0.010][loss_centermap: 0.067][loss_R:
0.017]
Time 00h 15m 52s, Epoch 09, Training finished , with mean training loss
0.08971253961324692
```

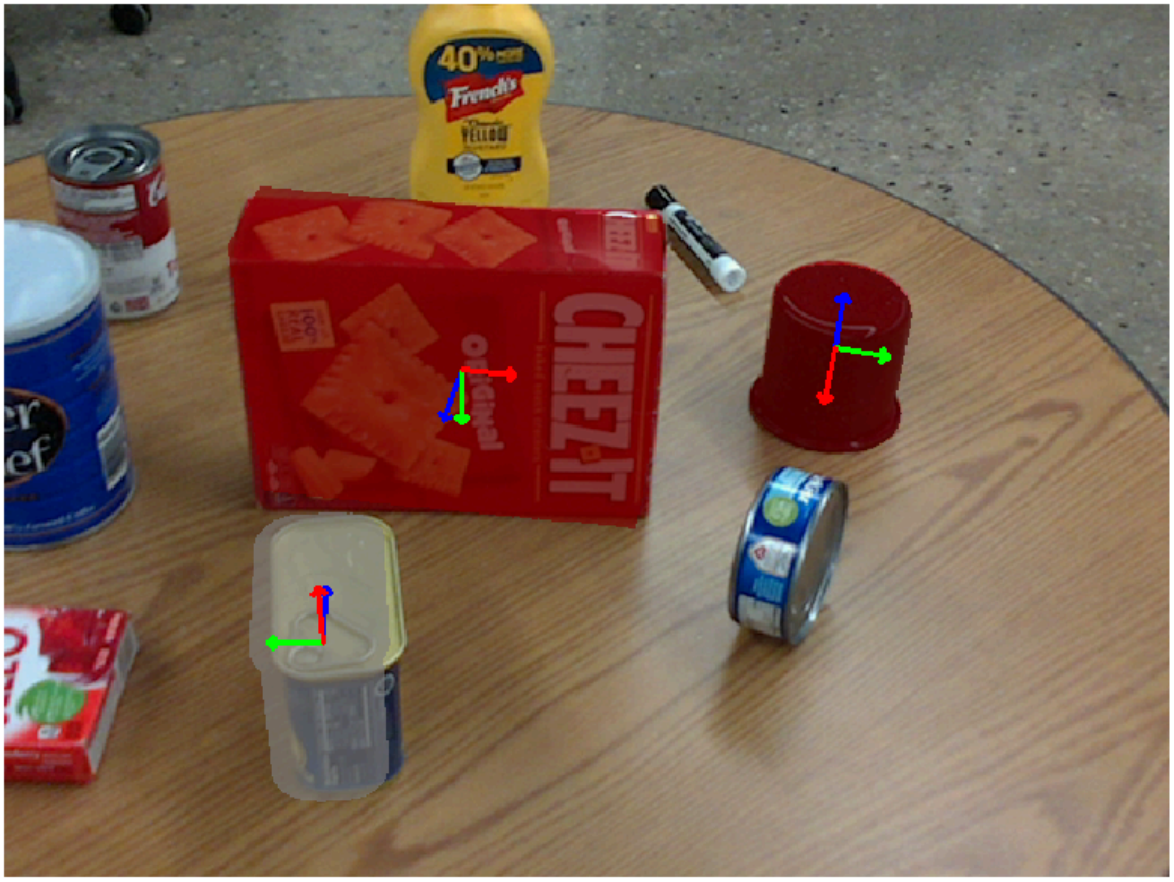


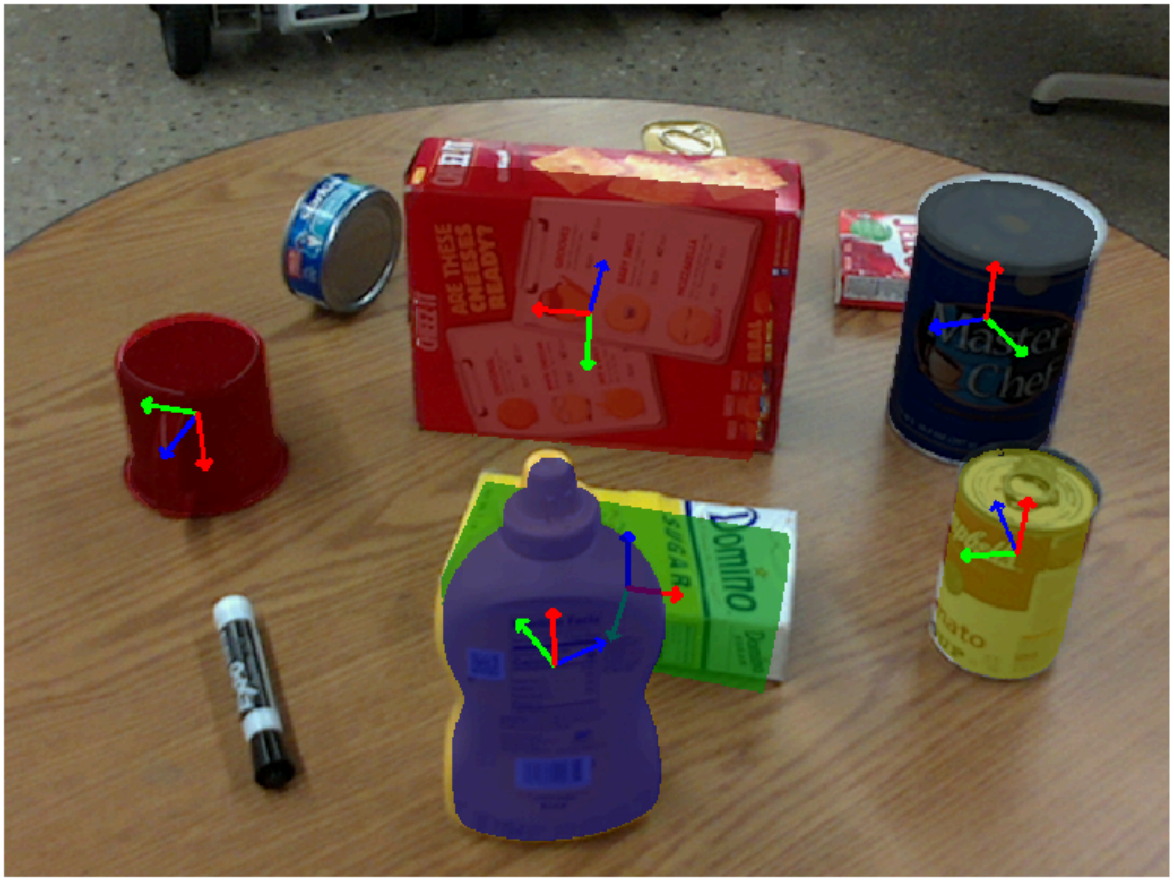
Inference

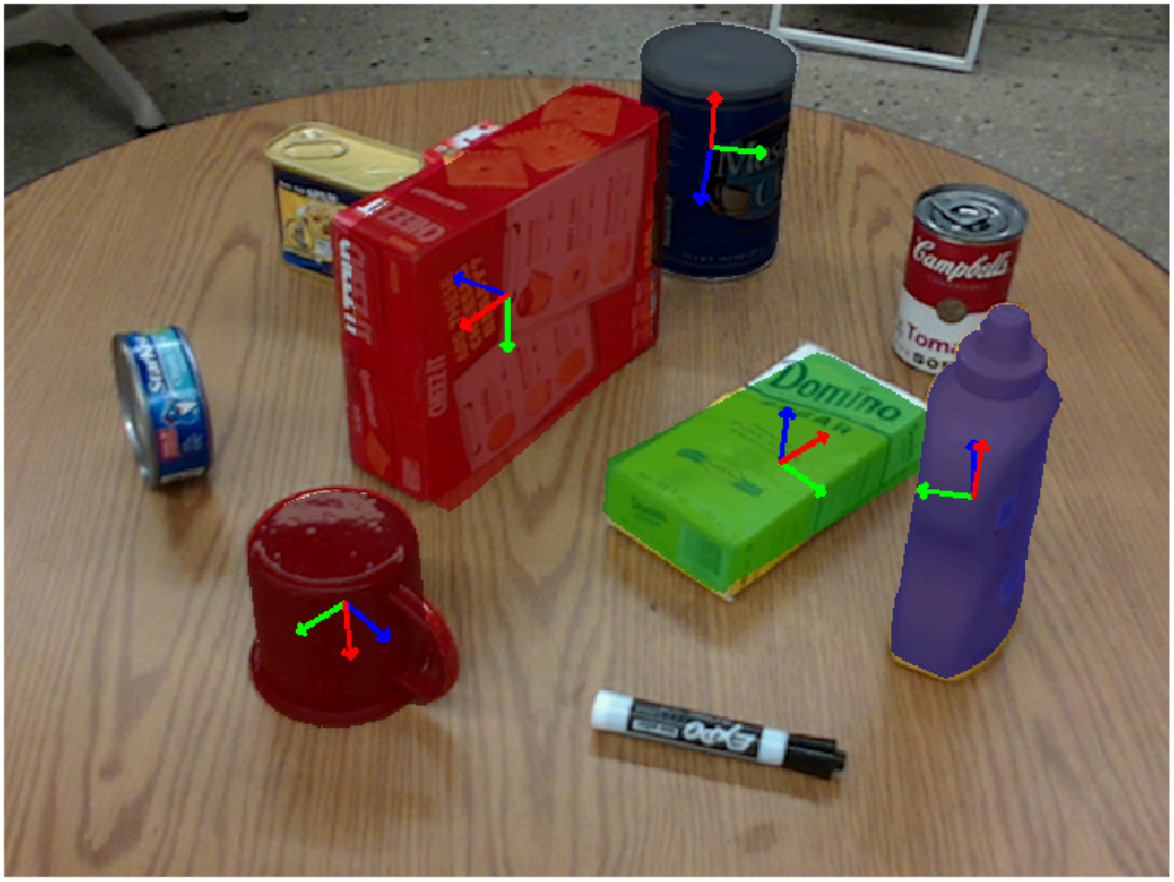
Visualize a few outputs from the full trained model. These could be improved if we used a larger model, trained for greater duration, and if we used ICP with depth data to refine the final estimates.

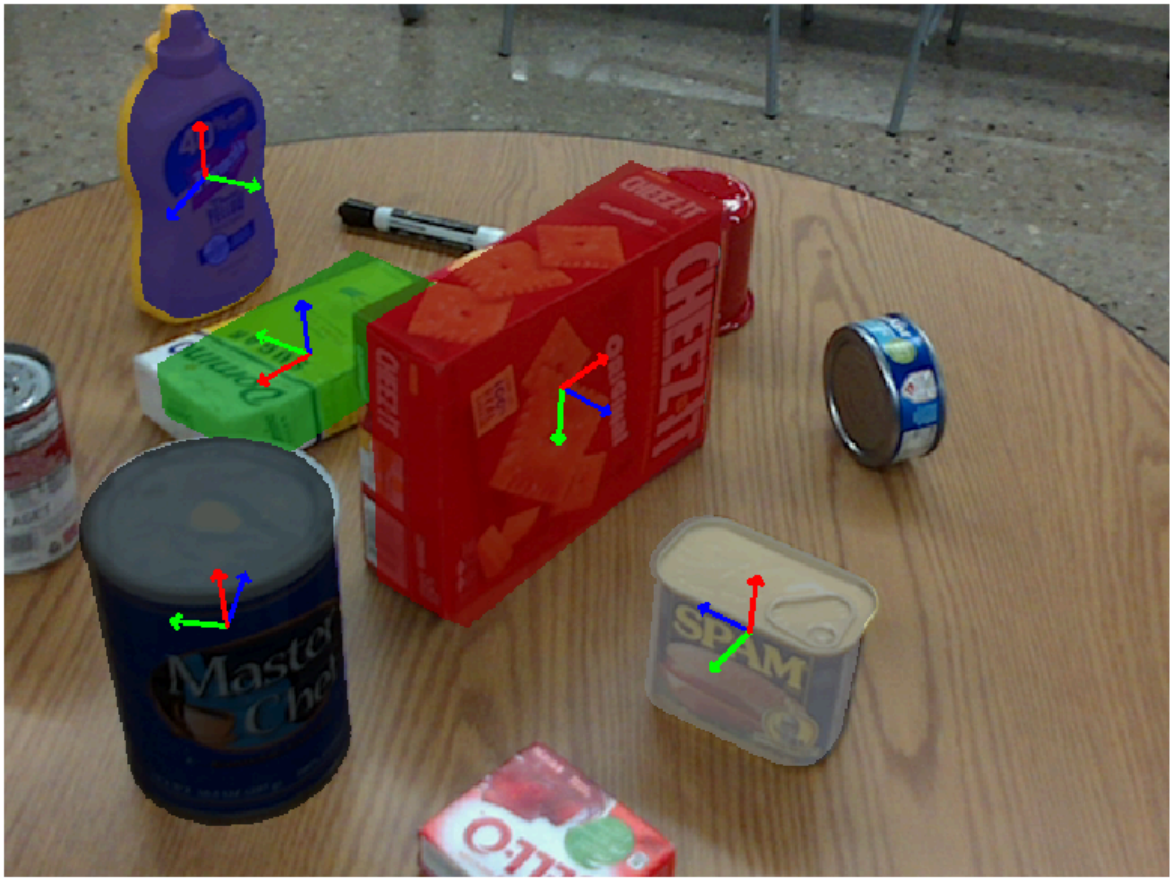
```
import os
import torch
from torch.utils.data import DataLoader
import torchvision.models as models

import utils
from pose_cnn import PoseCNN, eval

GOOGLE_DRIVE_PATH = '.'
utils.reset_seed(0)

dataloader = DataLoader(dataset=val_dataset, batch_size=BATCH_SIZE)

vgg16 = models.vgg16(weights=models.VGG16_Weights.IMAGENET1K_V1)
posecnn_model = PoseCNN(pretrained_backbone = vgg16,
                models_pcd = torch.tensor(val_dataset.models_pcd).to(DEVICE,
dtype=torch.float32),
                cam_intrinsic = val_dataset.cam_intrinsic).to(DEVICE)
posecnn_model.load_state_dict(torch.load(os.path.join(GOOGLE_DRIVE_PATH,
"posecnn_model.pth")))

num_samples = 5
for i in range(num_samples):
    out = eval(posecnn_model, dataloader, DEVICE)

    plt.axis('off')
    plt.imshow(out)
    plt.show()
```

```
/tmp/ipykernel_62113/1271061729.py:18: FutureWarning: You are using `torch.load`
with `weights_only=False` (the current default value), which uses the default
pickle module implicitly. It is possible to construct malicious pickle data which
will execute arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for
more details). In a future release, the default value for `weights_only` will be
flipped to `True`. This limits the functions that could be executed during
unpickling. Arbitrary objects will no longer be allowed to be loaded via this
mode unless they are explicitly allowlisted by the user via
`torch.serialization.add_safe_globals`. We recommend you start setting
`weights_only=True` for any use case where you don't have full control of the
loaded file. Please open an issue on GitHub for any issues related to this
experimental feature.
  posecnn_model.load_state_dict(torch.load(os.path.join(GOOGLE_DRIVE_PATH,
"posecnn_model.pth")))
```


40%
French's
CHEEZ-IT


ARE THESE CHEESES READY?
Domino
Master Chef


Campbells
Domino


CHEEZ-IT
SPAM

Finally, let's measure the quantitative accuracy of our trained model using the 5°5cm metric. That is, we'll count how many visible objects our model was able to predict correctly, where a correct prediction is defined as one with a rotation error of less than 5° and a translation error of less than 5cm.

The instructor's model, trained with the hyperparameters above achieves 29.3%.

```
%matplotlib inline
import math
import torch
from torch.utils.data import DataLoader
import torchvision.models as models

import pyquaternion
from tqdm import tqdm

import utils
from pose_cnn import PoseCNN

utils.reset_seed(0)

dataloader = DataLoader(dataset=val_dataset, batch_size=BATCH_SIZE)

posecnn_model.load_state_dict(torch.load(os.path.join(GOOGLE_DRIVE_PATH,
"posecnn_model.pth")))
posecnn_model.eval()


T_thresh = 5 # cm
```

```
R_thresh = 5 # deg

total =0
correct = 0
for batch in tqdm(dataloader):
    for item in batch:
        batch[item] = batch[item].to(DEVICE)
    pose_dict, segmentation = posecnn_model(batch)
    for bidx in range(BATCH_SIZE):
        objs_visib = batch['objs_id'][bidx].cpu().tolist()
        objs_preds = sorted(list(pose_dict[bidx].keys()))
        for objidx, objs_id in enumerate(objs_visib):
            if objs_id==0:
                continue

            total += 1
            if objs_id not in objs_preds:
                continue
            RT_pred = pose_dict[bidx][objs_id]
            RT_true = batch['RTs'][bidx][objidx].cpu().numpy()

            # Translation error
            T_pred = RT_pred[:3,3]
            T_true = RT_true[:3,3]
            T_err = 100*np.linalg.norm(T_pred-T_true) # error in cm

            # Rotation error
            R_true = pyquaternion.Quaternion(matrix=RT_true[:3,:3],atol=1e-6)
            R_pred = pyquaternion.Quaternion(matrix=RT_pred[:3,:3],atol=1e-6)

            R_rel = R_pred * R_true.conjugate
            R_err = math.degrees(R_rel.angle)

            if T_err<T_thresh and R_err<R_thresh:
                correct+=1

print("Accuracy at 5°5cm:",correct/total)
```

```
/tmp/ipykernel_62113/1024407368.py:17: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  posecnn_model.load_state_dict(torch.load(os.path.join(GOOGLE_DRIVE_PATH, "posecnn_model.pth")))
100%|██████████| 125/125 [01:25<00:00,  1.46it/s]

Accuracy at 5°5cm: 0.376227495908347
```